디자인 패턴

2026. 3. 17

미림 마이스터고

남궁성

seong.namkung@gmail.com

디자인 패턴 목록(1/3)

1. Singleton – 인스턴스를 단 하나만 생성(5) p.102
2. Flyweight – 같은 것을 공유해서 절약(20) p.364
3. Prototype – 복사해서 인스턴스를 생성(6) p.112
4. Template Method – 하위 클래스에서 구체적으로 처리(3) p.74
5. Strategy – 알고리즘을 모두 바꿈(10) p.186
6. Bridge – 기능 계층과 구현 계층을 나눔(9) p.172
7. Composite – 그릇과 내용물을 동일시(11) p.204
8. Decorator – 장식물과 내용물을 동일시(12) p.218
9. Command – 명령을 클래스로 표현(22) p.394

디자인 패턴 목록(2/3)

10. State – 상태를 클래스로 표현(19) p.340
11. Proxy – 필요할 때 만들기(21) p.380
12. Observer – 상태 변화를 통보(17) p.308
13. Memento – 상태를 저장(18) p.322
14. Factory Method - 하위 클래스에서 인스턴스를 생성(4) p.88
15. Abstract Factory – 부품을 조립해서 제품을 만듦(8) p.144
16. Builder – 복잡한 인스턴스를 조립(7) p.128
17. Adapter – 사이에 끼워 재사용(2) p.58

디자인 패턴 목록(3/3)

18. Iterator – 처리를 반복(1) p.42
19. Visitor – 데이터 구조를 돌아다니면서 처리(13) p.240
20. Interpreter – 문법 규칙을 클래스로 표현(23) p.414
21. Chain of Responsibility – 책임을 떠넘기기(14) p.260
22. Facade – 단순한 창구 만들기(15) p.274
23. Mediator – 중재자를 통해 처리(16) p.288

객체지향개념 5가지 원칙 - SOLID

- SRP – 단일 책임의 원칙. 하나의 메서드는 하나의 기능만
- OCP – 개방 폐쇄 원칙(상속 Open, 변경 Close). 변경x 상속o
- LSP – 리스코프 치환 원칙. 조상을 자손으로 대체 가능해야
- ISP – 인터페이스 분리 원칙. 다른 목적의 인터페이스는 분리
- DIP – 의존성 역전 원칙. 구체화 의존x, 추상화 의존o
  클래스가 아닌, 인터페이스에 의존하라.

-1. 디자인 패턴 공부의 시작

생성자의 단점

1. 항상 새로운 객체를 생성 -> factory 메서드, Flyweight패턴
2. 매개변수가 많아지면 구분 어려움 -> Builder패턴

```
public class UserTest {
    public static void main(String[] args) {
//        User user = new User("kim", 20, "kim@test.com");
        User user = new User.Builder()
                .name("kim")
                .age(20)
                .email("kim@test.com")
                .build();
    }
}
```

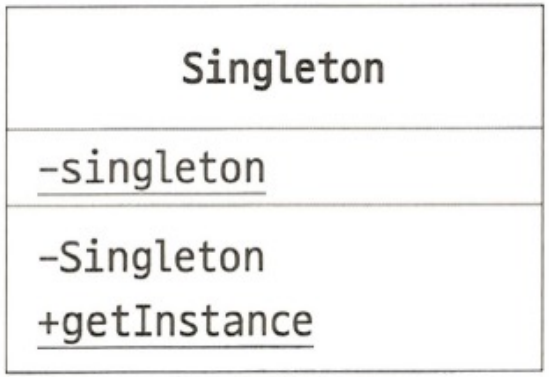
0. Factory method – 객체를 반환하는 메서드

```java
public class FactoryMethodTest {
    public static void main(String[] args) {
        Product p = new Product(); // 생성자
        Product p2 = Factory.getInstance(); // factory method
    }
}
```

```java
class Product {}  5 usages
class Factory {  1 usage
    static Product getInstance() {  1 usage
        return new Product();
    }
}
```

```java
class Product {}  6 usages
class Factory {  1 usage
    static Product p = new Product();  1 usage
    static Product getInstance() {  1 usage
        return p; // 미리 생성한 객체를 반환
    }
}
```

1. Singleton – 인스턴스를 단 하나만 생성

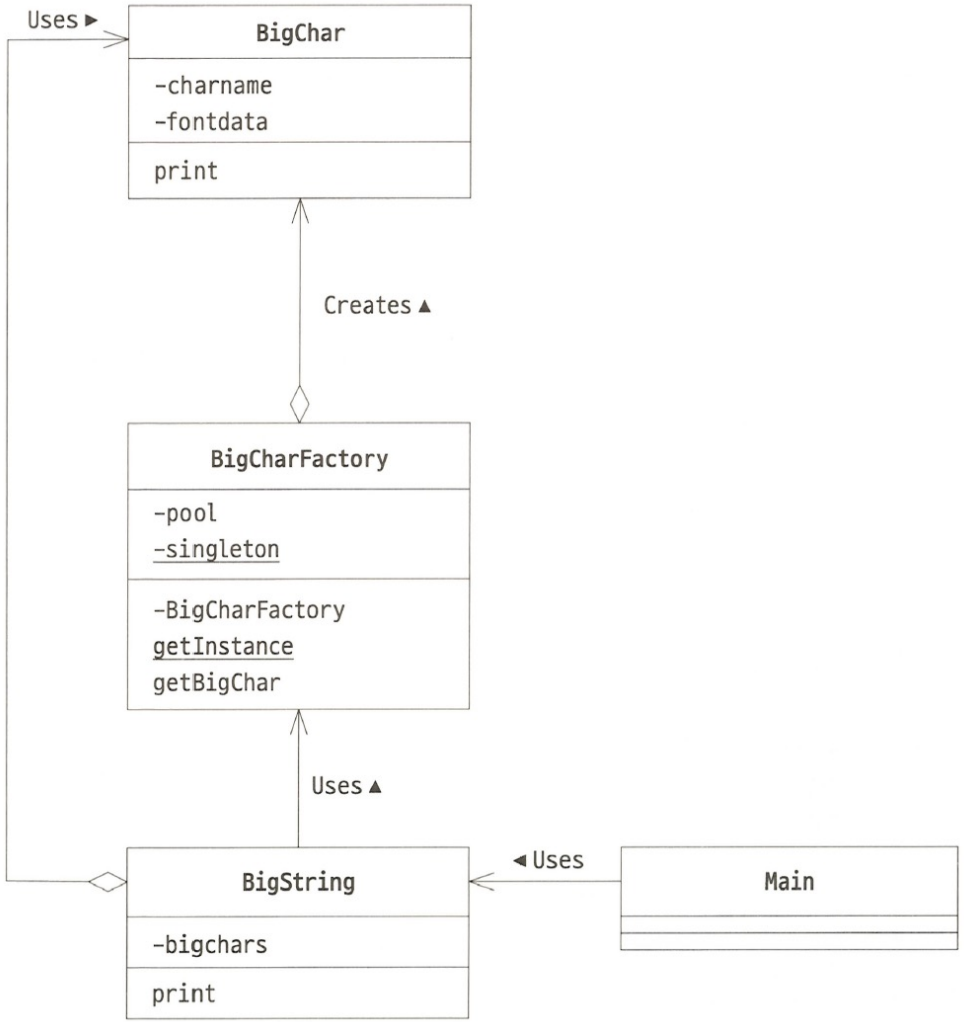


1. Singleton – 인스턴스를 단 하나만 생성

```java
public class Main {
    public static void main(String[] args) throws Exception {
        Singleton s  = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance();
        System.out.println("s = " + s);
        System.out.println("s2 = " + s2);
    }
}
```

```java
public class Singleton {
    static Singleton s = new Singleton();

    private Singleton() {}

    static Singleton getInstance() {
        return s;
    }
}
```

2. Flyweight – 인스턴스를 미리 생성해서 공유(n개)



2. Flyweight – 인스턴스를 미리 생성해서 공유(n개)

```java
public class Flyweight {
    private Map map = new HashMap();

    public void register(Object obj){
        String name = obj.getClass().getSimpleName();
        map.put(name, obj);
    }

    public Object getBean(Class clazz) throws Exception {
        for(Object obj : map.values())
            if(clazz.isInstance(obj))
                return obj;

        Object obj = clazz.newInstance();
        register(obj);
        return obj;
    }
}
```

```java
Flyweight f = new Flyweight();
f.register(new Car());

System.out.println(f.getBean(Car.class));
System.out.println(f.getBean(Car.class));
System.out.println(f.getBean(Engine.class));
System.out.println(f.getBean(Engine.class));
```

2. Flyweight – 인스턴스를 미리 생성해서 공유(n개)

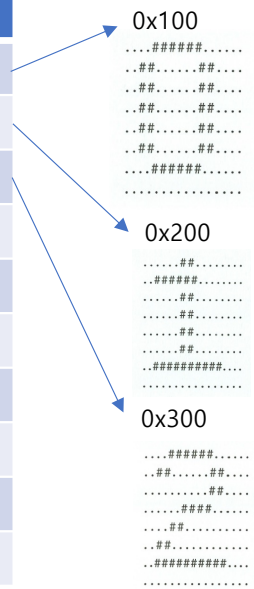
```java
public final class Integer extends Number
        implements Comparable<Integer>, Constable, ConstantDesc {
            //...
            private static final class IntegerCache {
                static final int low = -128;
                static final int high;

                @Stable
                static final Integer[] cache;
```

```java
public class IntegerTest {
    public static void main(String[] args) {
        System.out.println(new Integer( value: 1)==new Integer( value: 1)); // false
        System.out.println(Integer.valueOf( i: 1)==Integer.valueOf( i: 1)); // true
    }
}
```

2. Flyweight – 인스턴스를 미리 생성해서 공유(n개)

```java
class BigCharFactory {  5 usages
    private Map<String, BigChar> pool = new HashMap<>(); // 0~9의 객체를 미리 만들어서 저장
    private static BigCharFactory singleton = new BigCharFactory();  1 usage
    private BigCharFactory() {}  1 usage
    public static BigCharFactory getInstance() { return singleton; }  1 usage
    public BigChar getBigChar(char charname) {  1 usage
        BigChar bc = pool.get(String.valueOf(charname)); // pool에서 꺼내서 반환
        if (bc == null) {
            bc = new BigChar(charname);
            pool.put(String.valueOf(charname), bc);
        }
        return bc;
    }
}
```

charName	fontdata
"0"	0x100
"1"	0x200
"2"	0x300
"3"	0x400
"4"	0x500
"5"	0x600
"6"	0x700
"7"	0x800
"8"	0x900
"9"	0x1000




```java
class BigChar {  4 usages
    private char charName;  1 usage
    private String fontData;  no usages
    BigChar(char charName){  1 usage
        this.charName = charName;
    }
    // ...
}
```

```java
public class FlyweightTest {
    public static void main(String[] args) {
        BigCharFactory factory = BigCharFactory.getInstance();
        BigChar bigChar = factory.getBigChar( charname: '0');
        System.out.println(bigChar);
    }
}
```